

Лаборатори №1: UI автомат тест — Playwright

F.CSA313 — Программ хангамжийн чанарын баталгаа ба тест (2026)

Зорилго

Энэ лабораторийн ажлын зорилго нь орчин үеийн UI автоматжуулалтын хэрэгсэл болох Playwright-ийн тусламжтай функционал тестийг хэрхэн автоматжуулах талаар танилцахад оршино. Playwright төслийг үүсгэн тохируулж, веб хөтөчтэй ажиллах ажиллагааг автоматжуулсан тестийн скрипт бичиж, үр дүнг нь тайлагнана. Мөн өмнөх жилүүдэд ашиглаж байсан Selenium-тэй харьцуулж, аль алиных нь давуу талыг ойлгоно.

Яагаад Playwright вэ?

Selenium бол UI автоматжуулалтын анхдагч, өнөөг хүртэл өргөн хэрэглэгддэг хэрэгсэл. Харин Playwright (Microsoft, 2020) нь сүүлийн жилүүдэд индустрид хамгийн хурдацтай тархаж буй хэрэгсэл болсон. Гол ялгаанууд:

Шинж	Selenium	Playwright
Лиценз	Apache 2.0 — үнэгүй, нээлттэй эх	Apache 2.0 — үнэгүй, нээлттэй эх
Хүлээлт (waits)	Гараар explicit wait бичих шаардлагатай	Auto-wait — элемент бэлэн болтол автоматаар хүлээнэ
Driver	Хөтөч бүрд тусдаа driver суулгана	Хөтөчүүд нэг командаар суудаг
Тест бичих туслах	Selenium IDE	Codegen — үйлдлээ бичүүлж код болгодог
Дебаг	Лог, скриншот	Trace viewer — алхам бүрийн бичлэг, DOM snapshot
Хэл	Java, Python, C#, JS	JS/TS, Python, Java, C#

Лицензийн тухайд: аль аль нь Apache 2.0 лицензтэй, бүрэн үнэгүй, нээлттэй эхийн (open source) төслүүд тул сонголт нь зардлаас бус техникийн давуу талаас хамаарна. Энэ хичээлийн бодлого: лабораторийн ажилд аль болох ҮНЭГҮЙ, боломжтой бол НЭЭЛТТЭЙ ЭХИЙН хэрэгслүүдийг ашиглана. Ингэснээр та ямар ч байгууллагад, лицензийн төлбөрөөс үл хамааран хэрэглэж чадах үр чадвар эзэмшинэ.

Ажлын орчны бодлого

Энэ болон цаашдын бүх лабораторийн заавар Ubuntu Linux орчинд зориулагдана (багш macOS хэрэглэдэг — командууд бараг ижил). Windows-д зориулсан тусдаа заавар өгөхгүй: Windows дээр ажиллаж буй оюутан зааврыг өөрийн орчиндоо өөрөө тохируулж суралцана — энэ нь өөрөө инженерийн чухал дадал.

Яагаад ингэдэг вэ? Энэ нь vendor lock-in буюу нэг үйлдвэрлэгчээс хараат байдлаас татгалзаж буй бодлогын үргэлжлэл: нээлттэй эхийн хэрэгсэл сонгодгийн адил, нээлттэй, кросс-платформ орчинд ажиллаж сурснаар та тодорхой нэг компанийн үйлдлийн систем, төлбөртэй хэрэгслээс хамааралгүй үр чадвар эзэмшинэ. Индустрийн серверүүд, CI/CD орчин (GitHub Actions гэх мэт) дийлэнх нь Linux дээр ажилладаг тул Linux орчны дадлага шууд практик ач холбогдолтой. Зөвлөмж: Windows хэрэглэгчид

WSL2 (Windows Subsystem for Linux) суулгавал зааврыг бараг өөрчлөлтгүй дагаж болно — гэхдээ тохируулгыг нь өөрөө хийнэ.

Багшийн зөвлөмж: Программ хангамжийн инженер болохоор суралцаж буй хүнд WSL2-ээс ч илүү Linux-ийг dual boot-оор суулгаж, ЖИНХЭНЭ Linux орчинд өдөр тутам ажиллаж сурахыг зөвлөж байна. Ubuntu-г Windows-ийн хажууд dual boot-оор суулгах нь өөрөө маш сайн сургалт: диск хуваалт, boot процесс, үйлдлийн системийн суурь ойлголтыг практикаар эзэмшүүлнэ. Ирээдүйн мэргэжлийн орчин тань (сервер, cloud, CI/CD, Docker) Linux байх тул түүн дотор нь "амьдарч" сурсан инженер хол явна.

Алхам 1: Орчин бэлдэх

- Node.js (v18+) суулгасан байх. Шалгах: `node --version`
- Дараах командаар Playwright төсөл үүсгэнэ (асуултуудад default хариулт сонгож болно):

```
npm init playwright@latest my-first-test
cd my-first-test
npx playwright install
```

- Python сонгох бол: `pip install pytest-playwright && playwright install`
- Жишээ тест ажиллаж буйг шалгах:

```
npx playwright test
npx playwright show-report
```

Алхам 2: Тестийн скрипт бичих

Дараах үйлдлийг автоматжуулсан шинэ тест файл (`tests/mytest.spec.ts`) бичнэ:

- Веб хөтөч нээж тестлэх вебээ ачаалах. Дадлагын сайт: <https://www.saucedemo.com> (тест хийх зориулалттай нээлттэй демо дэлгүүр) эсвэл өөрийн сонгосон веб.
- Нэвтрэх үйлдэл гүйцэтгэх (saucedemo дээр: `standard_user` / `secret_sauce`).
- Элемент олохдоо Playwright-ийн орчин үеийн locator-уудыг ашиглах: `getByRole`, `getByLabel`, `getByText`, `getByTestId`. XPath-аас аль болох зайлсхий — яагаад гэдгийг тайландаа тайлбарла.

Жишээ (TypeScript):

```
import { test, expect } from '@playwright/test';

test('амжилттай нэвтрэх', async ({ page }) => {
  await page.goto('https://www.saucedemo.com');
  await page.getByPlaceholder('Username').fill('standard_user');
  await page.getByPlaceholder('Password').fill('secret_sauce');
  await page.getByRole('button', { name: 'Login' }).click();
  await expect(page.getByText('Products')).toBeVisible();
});
```

Алхам 3: Assertions — үр дүнг шалгах

- Амжилттай нэвтэрснийг `expect(...).toBeVisible()`, `toHaveURL()`, `toHaveText()` зэргээр шалга.
- Сөрөг тест нэм: бүрүү нууц үгээр нэвтрэхэд алдааны мессеж гарч буйг шалгах тест бич.
- Нийт дор хаяж 3 тест: амжилттай нэвтрэх, амжилтгүй нэвтрэх, нэвтэрсний дараах нэг үйлдэл (ж: бараа сагслах, хайлт хийх).

Алхам 4: Codegen ба Trace viewer турших

- Codegen-ээр тест бичүүлж үзэх: `npx playwright codegen saucedemo.com` — гарсан кодыг өөрийн бичсэнтэй харьцуул.
- Тестээ trace-тэй ажиллуулж, дүрс бичлэгийг нь үзэх:

```
npx playwright test --trace on  
npx playwright show-trace test-results/.../trace.zip
```

- Тестээ санаатайгаар унагаж (assertion-оо буруу болгож), trace viewer дээр алдааг хэрхэн мөшгихийг ажигла.

Алхам 5: Төгсгөх алхам

- Logout буюу гарах үйлдлээр тест бүрээ зөв төгсгө.
- Тест хоорондоо хамааралгүй, дангаараа ажилладаг байх ёстой (test isolation).

Алхам 6: GitHub-д байршуулж баримтжуулах

Энэ болон цаашдын бүх кодтой лабораторийн ажлыг ЗӨВХӨН GitHub репозиторийн линкээр хүлээж авна. Архивласан файл (zip, rar) хүлээж авахгүй.

- GitHub-д нээлттэй (public) репозитори үүсгэж, төслөө push хий. .gitignore ашиглан node_modules, test-results зэргийг оруулахгүй байх.
- GitHub анх удаа хэрэглэж байгаа бол өөрөө сүрч эзэмш — энэ нь энэ лабораторийн ГОЛ ШАЛГУУРЫН НЭГ. Орчин үеийн программ хангамжийн инженер Git/GitHub-гүйгээр ажиллах боломжгүй тул эхний лабораториос л сурсан байх ёстой. Эхлэх цэг: docs.github.com/get-started.
- Commit-уудаа ажлын явцад тогтмол хий — бүх ажлыг нэг commit-оор оруулсан түүх нь Git-ийг жинхэнэ утгаар нь ашиглаагүйг харуулна.
- Код дотроо коммент бичиж бүрэн тайлбарла.
- README.md-д: юу хийснээ, Playwright ба Selenium-ийн ялгааны талаарх өөрийн ажиглалтыг (5-10 өгүүлбэр) бич.
- HTML report эсвэл тест ажиллаж буй бичлэгийг (`npx playwright test --video on`) репозитортоо хадгал.

Илгээх журам (MS Teams)

- Teams дээрх тухайн Assignment-д GitHub репозиторийнхоо линкийг илгээнэ.
- **АНХААР** — Линкээ хавсаргасны дараа заавал "Turn in" товчийг дар! Хавсаргачихаад Turn in дараагүй ажил "илгээгдээгүй" төлөвтэй үлддэг тул үнэлэгдэхгүй.
- Assignment-ийн дуусах хугацааг (due date) урьдчилан харж, хаагдахаас ӨМНӨ илгээ.
- **АНХААР** — Хугацаа хаагдсаны дараа илгээсэн ажлыг хийгээгүйд тооцно — энэ дүрэм хатуу мөрдөгдөнө.

Нэмэлт даалгавар (AI өнцөг, заавал биш)

AI түслэх (Claude, ChatGPT, Copilot) ашиглан нэвтрэх хуудасны тестүүдийг үүсгүүлж үз. Дараа нь: (1) AI-ийн гаргасан тест таны бичсэнээс юугаараа ялгаатай байсан, (2) ямар алдаа, дутагдал байсан, (3) засаж

сайжруулсан эсэхээ тайландаа 5-8 өгүүлбэрээр дүгнэ. Заавал биш — гүйцэтгэсэн оюутны ажлыг идэвх, оролцооны үнэлгээнд тусгана. AI-г шүүмжгүй хуулбарлахгүй, үнэлж хэрэглэх нь энэ хичээлийн гол зарчим.

Үнэлгээ — тэнцсэн / тэнцээгүй

Лаб бүр 1 оноогоор, зөвхөн ТЭНЦСЭН / ТЭНЦЭЭГҮЙ гэж үнэлэгдэнэ. Доорх заавал шаардлагууд БҮГД хангагдсан бол тэнцсэнд тооцно; аль нэг нь дутуу бол тэнцээгүй. Хугацаа хаагдсаны дараа ирүүлсэн ажлыг хийгээгүйд тооцох ба нөхөх боломжгүй.

	Тэнцэх заавал шаардлага
☐	GitHub public репозиторийн линкийг Teams Assignment-д хугацаанд нь Turn in дарж илгээсэн
☐	Репозитори зөв бүтэцтэй: .gitignore (node_modules, test-results орсонгүй), ажлын явцыг харуулсан 3+ утга төгөлдөр commit
☐	Playwright төсөл орчинд суулгагдаж, npx playwright test амжилттай ажилладаг
☐	Дор хаяж 3 тест: амжилттай нэвтрэх, амжилтгүй нэвтрэх, нэвтэрсний дараах нэг үйлдэл — орчин үеийн locator (getByRole/getByLabel/getByText) ба assertion ашигласан
☐	Trace эсвэл HTML report / бичлэг репозиторид хавсаргасан
☐	README.md: юу хийснээ болон Playwright–Selenium харьцуулсан ажиглалт (5-10 өгүүлбэр); код comment бүхий